App router and rendering

14A. App Router & Rendering

Questions: 4 **Difficulty:** Mid to Senior **Companies:** Google, Amazon, Flipkart, Razorpay, Swiggy, Vercel

Q 14.1 How does the Next.js App Router work? Explain file-based routing, layouts, and loading/error boundaries.

Difficulty: Mid **Asked at:** Flipkart, Razorpay, Swiggy, Amazon

The App Router (introduced in Next.js 13, stable in 14+) uses file-system conventions to define routes, layouts, loading states, and error handling. No route configuration files — the folder structure IS the router.

The file conventions:

JAVASCRIPT Copy

```
app/
  ├── layout.tsx          ← Root layout (wraps entire app, includes <html>
  ├── page.tsx            ← Home page (/)
  ├── loading.tsx         ← Loading UI for / (shows while page.tsx loads)
  ├── error.tsx           ← Error UI for / (catches errors in page.tsx)
  ├── not-found.tsx       ← 404 page
  │
  ├── products/
  │   ├── layout.tsx      ← Products layout (shared sidebar, filters)
```

```
│   └── [slug]/
│       ├── page.tsx      ← Product detail page (/products/running-shoes)
│       └── error.tsx     ← Error boundary for product detail
│
├── checkout/
│   ├── page.tsx          ← Checkout page (/checkout)
│   └── success/
│       └── page.tsx      ← Success page (/checkout/success)
│
└── (auth)/               ← Route group (parentheses = no URL segment)
    ├── layout.tsx        ← Shared auth layout (centered card)
    ├── login/
    │   └── page.tsx      ← Login page (/login, not /auth/login)
    └── signup/
        └── page.tsx      ← Signup page (/signup)
```

Layouts persist across navigations. When a user navigates from `/products` to `/products/running-shoes`, the root layout and the products layout don't re-render. Only the inner `page.tsx` swaps. This means navigation is instant for shared layouts — the sidebar, header, and navigation stay mounted.

`loading.tsx` **creates automatic Suspense boundaries:**

TSX Copy

```tsx
// app/products/loading.tsx
export default function Loading() {
  return (
    <div className="grid grid-cols-3 gap-4">
      {Array.from({ length: 9 }).map((_, i) => (
        <div key={i} className="h-64 bg-gray-200 animate-pulse rounded-lg" />
      ))}
    </div>
  );
}
// This skeleton shows immediately while page.tsx fetches data
```

skeleton instantly, then swaps in the real content when data arrives.

`error.tsx` **creates automatic error boundaries:**

TSX Copy

```tsx
'use client';  // Error boundaries must be client components

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>Something went wrong loading products</h2>
      <p>{error.message}</p>
      <button onClick={reset}>Try again</button>
    </div>
  );
}
```

Errors in `page.tsx` are caught by the nearest `error.tsx` without crashing the entire app. The layout above stays intact — only the errored section shows the fallback.

Dynamic routes with `[slug]` capture URL parameters:

TSX Copy

```tsx
// app/products/[slug]/page.tsx
export default async function ProductPage({ params }) {
  const { slug } = await params;
  const product = await getProduct(slug);
  return <ProductDetails product={product} />;
}
```

Follow-up: "What are parallel routes and intercepting routes?"

Parallel routes (`@modal`, `@sidebar`) render multiple pages in the same layout simultaneously — useful for modals that have their own URL. Intercepting routes (`(.)photo/[id]`) catch a navigation and render it in a different context — like showing

a photo in a modal when clicked from a feed, but as a full page when accessed directly by URL.

Q 14.2 What's the difference between Server Components and Client Components in Next.js?

Difficulty: Mid **Asked at:** Every company asking about Next.js

Covered conceptually in Chapter 7 (Q7.6). Here's the practical Next.js application.

The default: Every component in the App Router is a Server Component unless you add `"use client"` at the top. This is the opposite of what most React developers expect.

Server Components can:

- Fetch data directly (database, file system, internal APIs)
- Use `async/await` at the component level
- Access backend resources without exposing them to the client
- Import large libraries without adding to client bundle

Server Components cannot:

- Use hooks (useState, useEffect, useContext)
- Use browser APIs (window, localStorage, navigator)
- Add event handlers (onClick, onChange)
- Use class components with lifecycle methods

TSX Copy

```tsx
// Server Component (default) — runs on server only
// This code never reaches the browser
import { db } from '@/lib/database';

export default async function DashboardPage() {
  const stats = await db.analytics.getStats();  // Direct DB access
```

```
  });

  return (
    <main>
      <StatsGrid stats={stats} />          {/* Server Component */}
      <OrderTable orders={recentOrders} />  {/* Server Component */}
      <LiveChat />                          {/* Client Component — needs i
    </main>
  );
}
```

TSX

Copy

```tsx
// Client Component — needs hooks and event handlers
'use client';

import { useState, useEffect } from 'react';

export default function LiveChat() {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState('');

  useEffect(() => {
    const ws = new WebSocket('/api/chat');
    ws.onmessage = (e) => setMessages(prev => [...prev, JSON.parse(e.data)
    return () => ws.close();
  }, []);

  return (
    <div>
      {messages.map(m => <p key={m.id}>{m.text}</p>)}
      <input value={input} onChange={e => setInput(e.target.value)} />
      <button onClick={() => sendMessage(input)}>Send</button>
    </div>
  );
}
```

button a Client Component.

TSX Copy

```tsx
// BAD: Entire page is a Client Component because of one interactive eleme
'use client';
export default function ProductPage() { ... }

// GOOD: Only the interactive piece is a Client Component
export default async function ProductPage() {
  const product = await getProduct();  // Server — no client JS
  return (
    <div>
      <h1>{product.name}</h1>            {/* Server — zero JS */}
      <p>{product.description}</p>        {/* Server — zero JS */}
      <AddToCartButton id={product.id} /> {/* Client — only this ships JS
    </div>
  );
}
```

Follow-up: "Can a Client Component import a Server Component?"

No, not directly. But a Client Component can receive Server Components as `children` or other props. This is how you compose them:

TSX Copy

```tsx
// This works — Server Component passed as children
<ClientLayout>
  <ServerRenderedContent />  {/* Rendered on server, passed as prop */}
</ClientLayout>
```

Q 14.3 How does data fetching work in Next.js App Router? Explain caching and revalidation.

In the App Router, data fetching happens directly in Server Components using `fetch` or any async function. No `getServerSideProps`, no `getStaticProps` — those are Pages Router patterns.

Basic data fetching:

TSX Copy

```tsx
// Fetched at request time by default
async function ProductPage({ params }) {
  const product = await fetch(`https://api.store.com/products/${params.slu

  return <ProductDetails product={product} />;
}
```

Controlling caching with `fetch` options:

TSX Copy

```tsx
// Static: cached forever (like SSG) — revalidate never
const data = await fetch('https://api.example.com/static', {
  cache: 'force-cache',
});

// Dynamic: never cached (like SSR) — fresh on every request
const data = await fetch('https://api.example.com/live', {
  cache: 'no-store',
});

// ISR: cached and revalidated every 60 seconds
const data = await fetch('https://api.example.com/products', {
  next: { revalidate: 60 },
});
```

On-demand revalidation — refresh specific data when it changes:

TSX Copy

```
import { revalidatePath, revalidateTag } from 'next/cache';

export async function POST(request) {
  const { path, tag } = await request.json();

  // Revalidate a specific page
  if (path) revalidatePath(path);

  // Revalidate all fetches with a specific tag
  if (tag) revalidateTag(tag);

  return Response.json({ revalidated: true });
}

// In your data fetching — tag the fetch
const products = await fetch('https://api.store.com/products', {
  next: { tags: ['products'] },
});

// When a product is updated, call:
// POST /api/revalidate { tag: 'products' }
// All pages using the 'products' tag are regenerated
```

Non-fetch data (database, ORM):

TSX Copy

```
import { unstable_cache } from 'next/cache';

const getCachedProducts = unstable_cache(
  async (category) => {
    return db.product.findMany({ where: { category } });
  },
  ['products'],              // Cache key prefix
  { revalidate: 3600 }       // Revalidate every hour
);
```

Follow-up: "How does Next.js deduplicate fetch requests?"

data in the component that needs it without worrying about redundant requests. The layout fetches the user, the sidebar fetches the user, the header fetches the user — one request total.

Q 14.4 What are Server Actions? How do they handle form submissions and mutations?

Difficulty: Senior **Asked at:** Razorpay, Flipkart, Swiggy, Amazon

Server Actions are async functions that run exclusively on the server but can be called directly from Client Components. They replace the need for API routes for simple mutations (create, update, delete operations).

TSX Copy

```tsx
// app/products/actions.ts
'use server';

import { db } from '@/lib/database';
import { revalidatePath } from 'next/cache';

export async function addToCart(productId: string) {
  const user = await getCurrentUser();
  await db.cart.create({
    data: { userId: user.id, productId, quantity: 1 },
  });
  revalidatePath('/cart');  // Refresh the cart page
}

export async function submitReview(formData: FormData) {
  const rating = Number(formData.get('rating'));
  const comment = formData.get('comment') as string;
  const productId = formData.get('productId') as string;

  await db.review.create({
    data: { productId, rating, comment },
  });
```

Using in a Client Component:

TSX Copy

```tsx
'use client';

import { addToCart } from './actions';

function AddToCartButton({ productId }) {
  async function handleClick() {
    await addToCart(productId);
    // Page automatically refreshes because the action called revalidatePa
  }

  return <button onClick={handleClick}>Add to Cart</button>;
}
```

Using with forms — progressive enhancement:

TSX Copy

```tsx
// This form works even before JavaScript loads (progressive enhancement)
import { submitReview } from './actions';

function ReviewForm({ productId }) {
  return (
    <form action={submitReview}>
      <input type="hidden" name="productId" value={productId} />
      <select name="rating">
        <option value="5">5 stars</option>
        <option value="4">4 stars</option>
        <option value="3">3 stars</option>
      </select>
      <textarea name="comment" placeholder="Write your review ... " />
      <button type="submit">Submit Review</button>
    </form>
```

The `action={submitReview}` attribute makes the form submit to the server action. If JavaScript hasn't loaded yet, the form submits as a standard POST request (progressive enhancement). If JavaScript is loaded, it submits via fetch without a page reload.

With useActionState for loading/error handling:

TSX Copy

```tsx
'use client';

import { useActionState } from 'react';
import { submitReview } from './actions';

function ReviewForm({ productId }) {
  const [state, formAction, isPending] = useActionState(submitReview, null

  return (
    <form action={formAction}>
      <input type="hidden" name="productId" value={productId} />
      <textarea name="comment" disabled={isPending} />
      <button type="submit" disabled={isPending}>
        {isPending ? 'Submitting ... ' : 'Submit Review'}
      </button>
      {state?.error && <p className="error">{state.error}</p>}
    </form>
  );
}
```

When to use Server Actions vs API Routes:

Server Actions: simple mutations (CRUD), form submissions, actions that revalidate cached data. They're simpler — no route to define, no HTTP method to handle, no request/response parsing.

API Routes: complex endpoints consumed by multiple clients (mobile app, third-party), webhooks from external services, long-running operations, WebSocket endpoints, or when you need specific HTTP semantics (status codes, headers, streaming).

Follow-up: "Are Server Actions secure? Can't someone call them directly?"

Server Actions are protected the same way API routes are — they run on the server, behind your authentication middleware. Next.js generates an endpoint for each action, but the client only sees an opaque reference. You still need to validate input and check authentication inside the action, just like you would in an API route.

QUICK CHECK — no marks, just for you.

1. A user navigates from /products to /products/running-shoes. Both the root layout and app/products/layout.tsx exist. What does the App Router re-render?

- ○ Both layouts re-render because the URL segment changed, then page.tsx renders
- ○ Only the inner page.tsx swaps; the root layout and products layout stay mounted
- ○ The products layout re-renders but the root layout is preserved
- ○ The whole tree re-renders from the root layout down, as with a full page load

Check answer

2. On one page, the root layout, the sidebar, and the header are all Server Components and each independently fetches the same URL https://api.store.com/user. How many actual HTTP requests does Next.js make?

- ○ One — Next.js deduplicates identical fetches during the render pass
- ○ Three — each component's fetch is a separate request unless you use cache: 'force-cache'
- ○ One only if you pass next: { tags: ['user'] } to every fetch call
- ○ Three on the first render, then one on subsequent renders once the cache is warm

Check answer

3. ...allbacks from an external payment provider. Based on the lesson's guidance, should this be a Server Action or an API Route, and why?

- An API Route — webhooks from external services need real HTTP semantics that Server Actions do not expose
- Either works identically; Server Actions are just API Routes with a nicer syntax
- A Server Action — Next.js generates a stable public endpoint for each action that external services can POST to
- A Server Action — it is a mutation that writes to the database and revalidates cached data

Check answer

Previous
Code Management

Next
Features and deployment

LEARN MORE

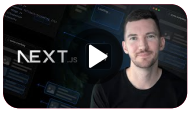
ARTICLES & DOCS

Next.js App Router (docs)
nextjs.org

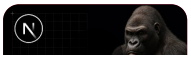
VIDEOS

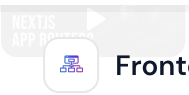


Next.js App Router explained simply
YouTube



App Router: routing, data fetching, caching
YouTube





App Router — what I wish I knew earlier

Frontend System Desi…

Hand-picked references for this lesson.